Звіт до лабораторної роботи № 5

Тема: Динамічно зв'язувані бібліотеки (DLL) в C/C++

1. Мета роботи

Вивчити принципи роботи з динамічно зв'язуваними бібліотеками (DLL) у Windows, навчитися коректно завантажувати та відвантажувати бібліотеки під час роботи програмного продукту, будувати об'єктно-орієнтовану архітектуру з розділенням класів по DLL, працювати з API LoadLibrary, GetProcAddress, FreeLibrary, а також аналізувати використання системних ресурсів при багатопоточних обчисленнях.

2. Архітектура програмного продукту

Програмний продукт складається з трьох окремих бінарних модулів: двох динамічних бібліотек (Interface.dll, Compute.dll) та головного виконуваного файлу (MainApp.exe). Абстрактний клас ICompute визначено один раз у спільному заголовному файлі Interface.h, який без змін підключається всіма трьома модулями - це і забезпечує сумісність розкладки віртуальної таблиці методів (vtable) класу Compute, реалізованого в окремій DLL, з тим, як цей клас використовує MainApp.exe.

Interface.dll (клас 1 - інтерфейс):

Містить лише оголошення абстрактного інтерфейсу ICompute (методи SetData, Run, GetResult, GetVersion, Release) та одну реально експортовану функцію GetInterfaceVersion(), яку MainApp.exe викликає статично. Оскільки MainApp.exe зібрано з посиланням на бібліотеку імпорту Interface.lib, операційна система завантажує Interface.dll автоматично в момент запуску процесу, ще до входу в main().

Compute.dll (клас 2 - реалізація обчислень):

Містить клас Compute, що реалізує ICompute: алгоритм повного перебору (брутфорс-перевірка кожного числа масиву на простоту методом пробних ділень до квадратного кореня), розподілений між заданою кількістю потоків std::thread; штучне навантаження пам'яті (виділення та торкання scratch-буфера, пропорційного розміру вхідних даних); обмеження максимально дозволеної кількості потоків. Ця DLL завантажується лише за потреби через LoadLibraryW і вивантажується після завершення обчислень через FreeLibrary, а перед завантаженням програма перевіряє, чи бібліотека вже не знаходиться в пам'яті, через GetModuleHandle.

MainApp.exe (головна програма):

Відповідає за послідовність дій: статичний виклик Interface.dll; перевірку, динамічне завантаження та подальше вивантаження Compute.dll; створення об'єкта обчислень через фабричну функцію CreateCompute(); запуск серії бенчмарків зі зміною кількості потоків і розміру вхідного масиву; паралельний запис показників використання ресурсів у CSV-файл; ранжування результатів за допомогою готової бібліотечної функції std::sort.

3. Структура розв'язання

Розв'язання у Visual Studio складається з трьох окремих проєктів в одному солюшені: двох проєктів типу «Dynamic-Link Library» (Interface, Compute) та одного проєкту типу «Console App» (MainApp). Нижче наведено призначення кожного модуля/файлу.

Модуль / файл	**Призначення**
Interface (DLL) Interface.h, InterfaceLib.cpp	Абстрактний інтерфейс ICompute, спільний для всіх трьох модулів, та єдина функція, яку реально експортує ця DLL - GetInterfaceVersion(). MainApp.exe лінкується з нею статично (завдання 2.1).
Compute (DLL) Compute.h, Compute.cpp	Конкретна реалізація ICompute: багатопотоковий алгоритм повного перебору (пошук простих чисел методом пробних ділень), штучне навантаження пам'яті, обмеження кількості потоків. Завантажується динамічно (завдання 1-2.2).
MainApp (EXE)	Точка входу: статично викликає Interface.dll, динамічно завантажує/вивантажує Compute.dll,

Модуль / файл	Призначення
main.cpp	запускає матрицю бенчмарків зі зміною кількості потоків і розміру масиву (завдання 2, 4.2).
MainApp ResourceMonitor.h / .cpp	Фоновий потік, що з заданим інтервалом записує обсяг пам'яті процесу (Psapi.h) та кількість потоків (TlHelp32) у CSV-файл (завдання 4.1).
MainApp ThirdPartyDemo.h / .cpp	Модуль підключення готової бібліотеки <algorithm>: сортування результатів бенчмарку за часом виконання через std::sort (завдання 3).

4. Інтерфейс ICompute та типи даних

Спільний абстрактний інтерфейс, визначений у Interface.h:

```
class ICompute
{
public:
   virtual void SetData(const std::vector<int>& data) = 0;
   virtual void Run(int threadsCount) = 0;
   virtual long long GetResult() = 0;
   virtual const char* GetVersion() = 0;
   virtual void Release() = 0;
protected:
   virtual ~ICompute() = default;
};

typedef ICompute* (*CreateCompute_t)();
```

Деструктор навмисно оголошено protected: зовнішній код не може викликати delete безпосередньо на вказівнику ICompute*, а зобов'язаний користуватися методом Release() - це гарантує, що об'єкт, створений усередині Compute.dll власним оператором new, буде знищений тим самим модулем (delete this всередині Compute::Release()), а не менеджером пам'яті MainApp.exe, що на межі DLL є потенційно небезпечним.

Тип	Призначення
ICompute*	Вказівник на абстрактний інтерфейс; єдиний тип, через який MainApp.exe звертається до об'єкта обчислень, не знаючи про існування класу Compute.
CreateCompute_t	Тип вказівника на функцію-фабрику; саме до цього типу приводиться результат GetProcAddress.
HMODULE	Дескриптор завантаженого модуля (DLL), який повертають LoadLibraryW і GetModuleHandleW.
std::vector<char> (m_memoryLoadBuffer)	Буфер штучного навантаження пам'яті всередині Compute; зберігається як поле класу, а не локальна змінна, щоб залишатися виділеним між викликами Run().
BenchmarkResult	Структура MainApp.exe з результатами одного прогону: розмір масиву, кількість потоків, час виконання, знайдена кількість простих чисел.

5. Методи та прийоми, використані при вирішенні задачі

1. Розділення інтерфейсу та реалізації по різних DLL: абстрактний клас ICompute живе в Interface.dll, а конкретний клас Compute - у Compute.dll; MainApp.exe працює лише з базовим вказівником ICompute*, не знаючи про існування класу Compute на етапі компіляції.
2. Статичне підключення Interface.dll: MainApp.exe лінкується з Interface.lib (бібліотекою імпорту), тому Interface.dll завантажується автоматично завантажувачем Windows разом із запуском процесу.
3. Динамічне підключення Compute.dll за патерном «перевір -> завантаж -> отримай адресу -> використай -> вивантаж»: GetModuleHandleW для перевірки, LoadLibraryW для завантаження, GetProcAddress для отримання фабричної функції, FreeLibrary для вивантаження після завершення роботи.

4. Фабричний патерн (Factory Method): замість публічного конструктора Compute.dll експортує функцію CreateCompute(), яка приховує сам клас Compute від MainApp.exe і повертає лише базовий інтерфейс ICompute*.
5. Багатопотокові обчислення: вхідний масив рівномірно розбивається на threadsCount неперервних діапазонів, кожен з яких обробляється власним std::thread; часткові результати потоків підсумовуються після join() усіх потоків.
6. Штучне навантаження пам'яті та обмеження кількості потоків: Run() виділяє scratch-буфер, пропорційний розміру вхідних даних, і обмежує запитану кількість потоків апаратно обґрунтованою верхньою межею (std::thread::hardware_concurrency(), помножене на коефіцієнт запасу).
7. Фонове логування ресурсів: ResourceMonitor запускає окремий std::thread, який з інтервалом 50 мс паралельно з обчисленнями дописує у CSV-файл поточний обсяг робочого набору пам'яті процесу та кількість його потоків, з міткою фази (розмір масиву; кількість потоків).
8. Використання готової бібліотечної функції (<algorithm>, std::sort) для ранжування результатів бенчмарку за часом виконання, замість написання власного алгоритму сортування.

6. Модуль підключення сторонньої бібліотеки

Згідно із завданням 3, у проєкті продемонстровано підключення не власних, а готових бібліотек у двох місцях:

1. <algorithm> (стандартна бібліотека C++): функція std::sort у модулі ThirdPartyDemo сортує вектор результатів бенчмарку за часом виконання; власний алгоритм сортування при цьому не писався.
2. Psapi.h та TlHelp32.h (бібліотеки Windows SDK): GetProcessMemoryInfo та CreateToolhelp32Snapshot/Thread32First/Thread32Next у модулі ResourceMonitor - готові системні засоби вимірювання пам'яті та кількості потоків процесу, без самостійної роботи з внутрішніми структурами ядра.

7. Розбір використаних функцій WinAPI

Нижче наведено розбір основних функцій WinAPI за шаблоном: тип поверненого значення; параметри та їх призначення; що виконує функція; особливості використання.

7.1. LoadLibraryW (динамічне завантаження DLL)

Синтаксис	HMODULE LoadLibraryW(LPCWSTR lpLibFileName);
Тип повернення	HMODULE - дескриптор завантаженого модуля, або NULL у разі помилки (деталі через GetLastError).
Параметри	lpLibFileName - ім'я або шлях до DLL-файлу (напр. L"Compute.dll").
Що виконує	Відображає вказану DLL у адресний простір поточного процесу, виконує її точку входу DllMain (якщо вона є) і повертає дескриптор для подальших викликів GetProcAddress.
Особливості	Якщо DLL вже завантажена іншою частиною процесу, функція лише збільшує лічильник посилань і повертає той самий дескриптор, а не завантажує модуль повторно.

7.2. GetProcAddress (отримання адреси функції)

Синтаксис	FARPROC GetProcAddress(HMODULE hModule, LPCSTR lpProcName);
Тип повернення	FARPROC - вказівник на знайдену функцію, або NULL, якщо функція з такою назвою не експортується модулем.
Параметри	hModule - дескриптор завантаженої DLL; lpProcName - ім'я експортованої функції у форматі ANSI-рядка (напр. "CreateCompute").
Що виконує	Шукає у таблиці експорту DLL функцію з вказаним іменем і повертає її адресу в пам'яті поточного процесу.

Синтаксис	FARPROC GetProcAddress(HMODULE hModule, LPCSTR lpProcName);
Особливості	Повернений тип FARPROC не збігається з реальною сигнатурою функції, тому результат обов'язково приводиться до конкретного типу вказівника на функцію (у роботі - до CreateCompute_t) перед викликом.

7.3. FreeLibrary / GetModuleHandleW

Синтаксис	BOOL FreeLibrary(HMODULE hLibModule); HMODULE GetModuleHandleW(LPCWSTR lpModuleName);
Тип повернення	FreeLibrary: BOOL - TRUE у разі успіху. GetModuleHandleW: HMODULE завантаженого модуля, або NULL, якщо модуль з такою назвою не завантажений у процес.
Параметри	hLibModule - дескриптор DLL, отриманий від LoadLibrary; lpModuleName - ім'я модуля для пошуку серед уже завантажених.
Що виконує	FreeLibrary зменшує лічильник посилань на DLL і вивантажує її з пам'яті процесу, коли лічильник сягає нуля. GetModuleHandleW лише перевіряє, чи модуль з заданим іменем уже присутній у процесі, не змінюючи лічильник посилань і не завантажуючи нічого нового.
Особливості	У роботі GetModuleHandleW використовується до LoadLibraryW (перевірка завантаження, завдання 2.2), а виклик після FreeLibrary - щоб підтвердити, що бібліотека дійсно вивантажена з пам'яті.

7.4. GetProcessMemoryInfo (обсяг пам'яті процесу)

Синтаксис	BOOL GetProcessMemoryInfo(HANDLE Process, PPROCESS_MEMORY_COUNTERS ppsmemCounters, DWORD cb);
Тип повернення	BOOL - TRUE у разі успіху, FALSE у разі помилки.
Параметри	Process - дескриптор процесу (у роботі - GetCurrentProcess()); ppsmemCounters - структура, яку функція заповнює лічильниками пам'яті; cb - розмір цієї структури в байтах.
Що виконує	Заповнює структуру PROCESS_MEMORY_COUNTERS поточними показниками використання пам'яті процесу, зокрема WorkingSetSize - обсягом фізичної пам'яті, яку процес реально займає в оперативній пам'яті в даний момент.
Особливості	Оголошена в Psapi.h і потребує лінкування з Psapi.lib (Visual Studio) або прапорця -lpsapi (MinGW); саме ця функція дає сирі дані для графіка використання RAM (завдання 4.1).

7.5. CreateToolhelp32Snapshot / Thread32First / Thread32Next (кількість потоків)

Синтаксис	HANDLE CreateToolhelp32Snapshot(DWORD dwFlags, DWORD th32ProcessID); BOOL Thread32First(HANDLE hSnapshot, LPTHREADENTRY32 lpte); BOOL Thread32Next(HANDLE hSnapshot, LPTHREADENTRY32 lpte);
Тип повернення	CreateToolhelp32Snapshot - дескриптор знімку системи, або INVALID_HANDLE_VALUE у разі помилки. Thread32First/Next - BOOL, TRUE поки є ще один запис про потік.

Синтаксис	HANDLE CreateToolhelp32Snapshot(DWORD dwFlags, DWORD th32ProcessID); BOOL Thread32First(HANDLE hSnapshot, LPTHREADENTRY32 lpte); BOOL Thread32Next(HANDLE hSnapshot, LPTHREADENTRY32 lpte);
Параметри	dwFlags - тип знімку (у роботі TH32CS_SNAPTHREAD - знімок усіх потоків усіх процесів системи); th32ProcessID - ігнорується для цього прапорця; lpte - структура THREADENTRY32, яку функції заповнюють інформацією про черговий потік, включно з th32OwnerProcessID.
Що виконує	Створює моментальний знімок усіх потоків, що існують у системі на момент виклику, після чого Thread32First/Next дозволяють перебрати цей знімок запис за записом.
Особливості	Windows не надає прямої функції «кількість потоків процесу», тому підрахунок виконується вручну: перебираються всі потоки системи, і рахуються лише ті, чий th32OwnerProcessID збігається з GetCurrentProcessId(); дескриптор знімку обов'язково закривається через CloseHandle після використання.

8. Налаштування проєкту та компіляція

Розв'язання зібрано в середовищі Visual Studio як один солюшен із трьома проєктами:

1. Interface - проєкт типу «Dynamic-Link Library (.dll)», містить Interface.h і InterfaceLib.cpp.
2. Compute - проєкт типу «Dynamic-Link Library (.dll)», містить Compute.h і Compute.cpp; у налаштуваннях проєкту (Additional Include Directories) додано шлях до Interface.h.

3. MainApp - проєкт типу «Console App (.exe)», містить main.cpp, ResourceMonitor.h/.cpp, ThirdPartyDemo.h/.cpp; у властивостях проєкту додано посилання (Project Dependencies / References) на проєкт Interface, щоб лінкер підхопив Interface.lib, та шлях до Interface.h в Additional Include Directories.

Оскільки Windows шукає DLL поруч із .exe-файлом, після збірки файли Interface.dll і Compute.dll було скопійовано у вихідний каталог MainApp.exe (напр. вручну або через подію post-build "xcopy" у властивостях проєкту MainApp). Без цього кроку LoadLibraryW і неявне завантаження Interface.dll завершувалися б помилкою, оскільки операційна система не знайшла б файли бібліотек.

1. Precompiled Headers довелось вимкнути вручну (Not Using Precompiled Headers) для обох DLL-проєктів, оскільки за замовчуванням VS створює шаблон з pch.h
2. Автоматично згенеровані VS файли (dllmain.cpp, pch.h, pch.cpp, framework.h, MainApp.cpp) були видалені з проєктів і замінені власними файлами
3. Additional Include Directories налаштовано для Compute і MainApp (шлях до папки Interface), для самого Interface — не потрібно
4. Статичне лінкування MainApp з Interface виконано через Add → Reference (а не вручну через Additional Dependencies)
5. DLL-файли (Interface.dll, Compute.dll) скопійовано вручну в папку з MainApp.exe, оскільки за замовчуванням VS кладе кожен проєкт у власну папку виводу

Коректність коду додатково перевірено крос-компіляцією під Linux компілятором MinGW-w64 (x86_64-w64-mingw32-g++, стандарт C++17):

```
x86_64-w64-mingw32-g++ -std=c++17 -Wall -Wextra -O2 -shared \
    -o Interface.dll InterfaceLib.cpp -Wl,--out-implib=libInterface.a
```

```
x86_64-w64-mingw32-g++ -std=c++17 -Wall -Wextra -O2 -shared -I../Interface \    -o
Compute.dll Compute.cpp -Wl,--out-implib=libCompute.a
```

```
x86_64-w64-mingw32-g++ -std=c++17 -Wall -Wextra -O2 -I../Interface \
    -o MainApp.exe main.cpp ResourceMonitor.cpp ThirdPartyDemo.cpp \
    -L../Interface -lInterface -lpsapi
```

Усі три модулі зібралися без помилок і без попереджень (з увімкненими прапорцями -Wall -Wextra), що підтверджує синтаксичну коректність коду та відповідність сигнатур викликів WinAPI.

9. Результати виконання

```
==================================================
   DLL demo: Interface.dll + Compute.dll + MainApp
==================================================

[Interface.dll] Interface.dll v1.0 (ICompute abstract interface)

Checking whether Compute.dll is already loaded...
Compute.dll is not loaded yet.
Loading Compute.dll with LoadLibraryW...
[Compute.dll]   Compute.dll v1.0 (brute-force primality counter)

Running benchmark matrix (2 data sizes x 4 thread counts)...
  size=1000000 threads=1 time=58.6036 ms primes=102531
  size=1000000 threads=2 time=21.3564 ms primes=102531
  size=1000000 threads=4 time=18.5741 ms primes=102531
  size=1000000 threads=8 time=16.9741 ms primes=102531
  size=4000000 threads=1 time=222.325 ms primes=409770
  size=4000000 threads=2 time=84.4186 ms primes=409770
  size=4000000 threads=4 time=62.6935 ms primes=409770
  size=4000000 threads=8 time=52.0588 ms primes=409770

Ranking (fastest -> slowest), via std::sort from <algorithm>:
  1. size=1000000 threads=8 time=17.0 ms primes=102531
  2. size=1000000 threads=4 time=18.6 ms primes=102531
  3. size=1000000 threads=2 time=21.4 ms primes=102531
  4. size=4000000 threads=8 time=52.1 ms primes=409770
  5. size=1000000 threads=1 time=58.6 ms primes=102531
  6. size=4000000 threads=4 time=62.7 ms primes=409770
  7. size=4000000 threads=2 time=84.4 ms primes=409770
  8. size=4000000 threads=1 time=222.3 ms primes=409770

Compute.dll after FreeLibrary: unloaded

Resource log written to "resource_log.csv".
Open it in Excel / Google Sheets / Python and build charts of
working_set_mb and thread_count over elapsed_ms for each phase
(task 4.1).

C:\Users\Admin\source\repos\lab5(1)\x64\Debug\MainApp.exe (process 7268) exited with code 0 (0x0).
```

Фрагмент згенерованого файлу resource_log.csv під час тестування:

	A	B	C	D	E
1	elapsed_ms	phase	working_se	thread_count	
2	0	size=10000(	73,8711	5	
3	0	size=10000(	74,0391	6	
4	0	size=10000(	74,0781	4	
5	0	size=10000(	74,1367	7	
6	0	size=40000(	280,031	4	
7	90	size=40000(	280,047	5	
8	0	size=40000(	280,031	4	
9	0	size=40000(	280,031	4	
10	0	size=40000(	280,02	4	
11	0	size=10000(	73,8789	5	
12	0	size=10000(	74,0352	4	
13	0	size=10000(	74,0781	7	
14	0	size=10000(	74,0156	11	
15	0	size=40000(	280,027	4	
16	94	size=40000(	280,047	5	
17	0	size=40000(	280,027	4	
18	0	size=40000(	280,027	4	
19	0	size=40000(	280,016	4	
20					

Для побудови графіків файл resource_log.csv відкривається в Excel (або Google Sheets / Python+matplotlib); стовпець phase дозволяє відфільтрувати рядки по конкретній комбінації «розмір масиву - кількість потоків», після чого для working_set_mb і thread_count будуються окремі лінійні графіки відносно elapsed_ms.

10. Аналіз використання ресурсів

1. Використання пам'яті (working_set): Споживання оперативної пам'яті демонструє чітку пропорційну залежність від розміру масиву. При size=100000 виділена пам'ять становить ~73.8–74.1 МБ, а при збільшенні навантаження в 4 рази (size=400000) обсяг пропорційно зростає до ~280.0 МБ і залишається стабільним до кінця фази.
2. Час виконання (elapsed_ms): На малих обсягах обчислення відбуваються майже миттєво (0 мс). Зі зростанням розміру буфера до size=400000 час обробки фіксується на рівні 90–94 мс.
3. Активність потоків (thread_count): Базовий стан системи становить 4–5 потоків (потоки процесу та моніторингу). Під час виконання обчислювальних фаз кількість потоків динамічно зростає, досягаючи пікових значень у 7–11 потоків, після чого після join() знову повертається до базового рівня.

11. Висновок

У ході виконання лабораторної роботи вивчено принципи роботи з динамічно зв'язуваними бібліотеками у Windows. Розроблено програмний продукт з об'єктно-орієнтованою архітектурою, розділеною на три модулі: Interface.dll (абстрактний інтерфейс ICompute, підключається статично), Compute.dll (багатопотокова реалізація алгоритму повного перебору з підтримкою штучного навантаження пам'яті, завантажується динамічно через LoadLibrary/GetProcAddress/FreeLibrary) та MainApp.exe (головна програма, що керує обома бібліотеками й логує використання ресурсів). Продемонстровано підключення готових бібліотек: стандартної <algorithm> (std::sort) та системних Psapi.h і TlHelp32.h для вимірювання пам'яті й кількості потоків процесу. У процесі роботи закріплено практичні навички проєктування архітектури на основі DLL, фабричного патерна для приховування конкретної реалізації за абстрактним інтерфейсом, а також аналізу використання системних ресурсів при багатопоточних обчисленнях.